ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name:**
- **Chinese Name:** 李羽姿
- **Email:** mlee33710@gmail.com
- **Student ID:** 112020028
- **University:** National Tsing Hua University

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594 documents.
2. (1) Raw text data is often large, heterogeneous, and noisy. Isolating a specific category reduces the number of documents and narrows the vocabulary, so frequent pattern mining runs more efficiently and the patterns it finds are relevant to a single domain.
   (2) If all categories are mined together, domain-specific patterns can be diluted. Frequent pattern mining keeps only itemsets that meet a minimum support threshold. A medical word combination such as {doctor, patient} may be frequent within sci.med but too rare across all 2,257 documents to pass the threshold. At the same time, words shared across topics would dominate the results. Filtering to sci.med first ensures that the discovered patterns reflect meaningful associations in the medical domain.

AI used for this answer: Yes

If yes, how: I used Claude to understand the assignment and the pandas filtering syntax.

External references (links):

- pandas documentation, Boolean indexing: https://pandas.pydata.org/docs/user_guide/indexing.html#boolean-indexing
- GeeksforGeeks, "Frequent Pattern Mining in Data Mining": https://www.geeksforgeeks.org/dsa/frequent-pattern-mining-in-data-mining/

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: Dropping these rows would be a flawed strategy for two reasons. First, we would lose 40% of the data, even though the text in these rows is still complete and useful.

Second, if the labels are not missing at random (for example, if one category is missing labels more often than others), the remaining 60% would no longer represent the original distribution, which introduces bias into later analysis.

An objective alternative is to train a text classifier on the 60% of labeled documents and use it to predict the missing labels for the other 40%. This approach, known as pseudo-labeling or self-training, is objective because the labels come from a consistent and reproducible model rather than manual judgment. Its accuracy can be checked on a validation set held out from the labeled data.

AI used for this answer: Yes

If yes, how: Concept explanation about Q2 before drafting. The reasoning in the answer was written by me.

External references (links):

- scikit-learn, "Semi-supervised Classification on a Text Dataset": https://scikit-learn.org/stable/auto_examples/semi_supervised/plot_semi_supervised_newsgroups.html
- scikit-learn User Guide, Semi-supervised learning: https://scikit-learn.org/stable/modules/semi_supervised.html

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No. The category proportions in the sample are very close to those in the original dataset, but they do not match exactly. This small discrepancy comes from sampling variability, which is inherent to random sampling.
2. With simple random sampling, every record has the same probability of being selected, so the class proportions are preserved only in expectation, not in any particular sample. In a highly imbalanced dataset, the expected number of minority-class records in a sample is small, so random fluctuation can cause the minority class to be severely under-represented or even missing entirely. As a result, a model trained or evaluated on such a sample may fail to learn or correctly assess the minority class.

AI used for this answer: Yes

If yes, how: Concept explanation (sampling variability and the risk of random sampling on imbalanced data).

External references (links):

- scikit-learn train_test_split (stratify parameter): https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. Increase. When stop_words=None, CountVectorizer includes common words (such as "the," "is," and "and") in the vocabulary. The shape changes from (2257, 35482) to (2257, 35788), adding 306 new columns.
2. Sparsity decreases slightly. I ran the code in the notebook: the percentage of zeros is 99.6772% with stop_words='english' and 99.5470% with stop_words=None. Because stopwords appear in almost every document, most cells in the 306 new columns are non-zero, which lowers the proportion of zeros. However, these columns are few compared with the total vocabulary of over 35,000 words, so the matrix remains highly sparse.

AI used for this answer: Yes

If yes, how: Concept explanation (how stopwords affect sparsity) and the code to compute sparsity (1 - nnz / total cells). I ran the code myself to obtain the numbers.

External references (links):

- CountVectorizer API document：https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
- Text feature extraction：https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1. The vocabulary shrinks sharply as the threshold increases. At 0.0, all 35,482 terms are kept, because every term appears at least once and therefore has a variance above zero. At 0.01, only 5,089 terms remain, and at 0.05, only 1,491 terms remain.
2. Words with very low document frequency, meaning those that appear in only one or a few documents, have variance close to zero and are eliminated first. For example, "zipped," "zimmerman," and "zus" each have a variance of about 0.000443.
3. A near-zero variance feature is almost constant (zero in nearly every document), so it provides little information to distinguish documents or categories. Rules learned from words that appear only once or twice also do not generalize to new data. Removing them therefore reduces dimensionality and computation with little loss in accuracy.

Threshold	**Number of terms kept (keep == True)**
0.0	kept = 35482
0.01	kept = 5089
0.05	kept = 1491

AI used for this answer: Yes

If yes, how: Concept explanation (how variance relates to document frequency and why low-variance features can be removed). I ran the code myself to obtain all the numbers.

External references (links):

- scikit-learn User Guide, "Removing features with low variance": https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. Neither list contains generic stop words, since they were removed by stop_words='english'. However, the two lists differ. The Pearson top 10 is mixed. Only three words are graphics terms (vga, video, card), while the others are conversational words (thanks, hi, looking, keywords, host) or likely header metadata (writes and people). The Spearman top 10 is mostly domain-specific: 7 of the 10 words, such as graphics, files, image, 3d, program, file, and windows, are related to graphics. Both lists also contain negatively correlated words, such as people, which appears more often in the other category.
2. Pearson: $r = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i-\bar{y})^2}}$; Spearman: $\rho = r(rank(x), rank(y))$. Pearson uses the raw values directly, so it measures the linear relationship between x and y and is sensible to extreme values. Spearman replaces each value with its rank and then applies the Pearson formula, so it measures a monotonic relationship.
3. It happens when a word appears consistency across documents of the topic, but its counts vary greatly, with a few documents containing very high counts. For example, "graphics" has the highest Spearman correlation with comp.graphics (0.521), but its Pearson correlation is only 0.178, so it does not appear in the Pearson top 10. In comp.graphics documents, its mean count is 1.034, but the maximum is 105.

AI used for this answer: Yes

If yes, how: Concept explanation (the difference between Pearson and Spearman correlation, illustrated with a small numerical example of how an extreme count lowers Pearson but not Spearman). I ran the code for comp.graphics myself and filled in the answers using my own notebook results.

External references (links):

- SciPy documentation, pearsonr: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html
- SciPy documentation, spearmanr: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Decreasing minSup from 9 to 3 greatly increases both the runtime and the number of patterns. For alt.atheism, the number of maximal patterns grows from 710 to 9547, and the runtime grows from 0.164s to 3.386s. Besides, I add one code of line to know the longest pattern length in the cell. The longest pattern also grows from 9 to 69 items. A lower threshold allows more items and combinations to pass, so the search space expands rapidly.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup fixes the threshold and lets the number of patterns vary, while k fixes the output size and lets the threshold determined by the data. I add two lines of codes to show the number of patterns and the lowest support among top-k to myself. In my results, Top-K FAE always returned exactly k patterns, and the lowest support among them dropped from 21 (k=100) to 16 (k=500). In other words, FAE effectively chosses the support threshold that keeps exactly k patterns, which is useful when we do not know minSup in advance.

Answer (3) — Maximal vs standard frequent patterns:

A maximal frequent pattern is a frequent pattern that has no frequent superset. Because every subset of a frequent pattern is also frequent, listing only the maximal patterns implicitly represents all frequent patterns without repeating their subsets. For example, if {i, j, k} is frequent, its 7 non-empty subsets are also maximal list includes only {i, j, k}. The trade-off is that the exact support of each subset is no longer reported.

AI used for this answer: Yes

If yes, how: Concept explanations of how k differs from minSup. I ran all experiments myself and filled in the answers using my own results.

External references (links):

- PAMI GitHub repository: https://github.com/UdayLab/PAMI
- mlxtend documentation, "fpmax: Maximal itemsets via the FP-Max algorithm": https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/fpmax/
- Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. *SIGMOD Record*, 29(2), 1–12. https://doi.org/10.1145/335191.335372

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. The full X_counts matrix has shape (2257, 35482), so each document is a point in a 35482-dimensional space, while a scatter plot can only show 2 axes (or 3 in a 3d plot). There is no direct way to draw a point that has 35482 coordinates on a flat plane.
   Choosing two words as the axes would not solve this. There are about millions of possible word pairs, and any single pair keeps only 2 of the 35482 dimensions and discards the rest. In addition, the matrix is 99.68% zeros (computed in my notebook), so for almost any two words, most documents would have a count of zero for both and would collapse onto the origin (0, 0). The plot would show one overlapping cluster rather than separate groups by category.
2. Dimensionality reduction algorithms must be applied first. The helpers folder (visualization_helpers.py) imports three of them: PCA from sklearn.decomposition, t-SNE and UMAP are non-linear methods that try to keep documents that are close in the original space close in the reduced space.

AI used for this answer: Yes

If yes, how: Concept explanation of why high-dimensional, sparse data cannot be plotted directly, and help locating the dimensionality reduction methods in the helpers folder. I verified the matrix shape (2,257 × 35,482) and sparsity (99.68%) by running code myself.

External references (links):

- Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065), 20150202. https://doi.org/10.1098/rsta.2015.0202
- van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605. https://jmlr.org/papers/v9/vandermaaten08a.html
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for dimension reduction. *arXiv preprint* arXiv:1802.03426. https://arxiv.org/abs/1802.03426

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE shows the most distinct cluster islands. For example, sci.med forms a dense, separate island in all three t-SNE plots, and alt.atheism and soc.religion.christian occupy different regions, although some documents from all categories still mix near the center. UMAP shows partial separation but more overlap. PCA shows the heaviest overlap: in all three filtering methods, almost all documents are squeezed near the origin, with only a few outlier documents pulled far away.
2. I learned that PCA is a linear method that projects data onto the directions of greatest variance. In sparse text data, these directions are dominated by a few very long documents with extremely high word counts, which appear as outliers far from the origin in my PCA plots, rather than by differences between categories. The pattern features are binary (0/1) and have small variance, so they barely affect PCA, which is why the three PCA plots look almost identical. In contrast, t-SNE and UMAP are non-linear methods that preserve local neighborhoods: documents that use similar words stay close together, so documents from the same category from clusters.
3. I thought that t-SNE preserves local neighborhoods but not global distances. It keeps similar documents close together, but the distance between separate clusters, as well as cluster size and density, is not meaningful. The layout also changes with parameters such as the random seed. In my plots, the dense sci.med island appears in different position in each t-SNE (top-left, left, and bottom), even though it contains similar documents. Therefore, a large gap between two clusters does not mean the categories are more different.

AI used for this answer: Yes

If yes, how: Concept explanations of why PCA (linear, variance-based).

External references (links):

- Wattenberg, Viégas & Johnson (2016), "How to Use t-SNE Effectively", *Distill* https://distill.pub/2016/misread-tsne/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. When I applied mib.transform to X['category'], the output had shape (2257, 4), so 4 binary columns were created, one for each category. mib.classes_ returned [0, 1, 2, 3], meaning the columns correspond to alt.atheism, comp.graphics, sci.med, and soc.religion.christian in that order. For example, the first document in my dataset belonds to comp.graphics, so it is represented as [0, 1, 0, 0].
2. From my perspective, the algorithm treats the code as real numbers, so it falsely assumes the categories have a meaningful order and equal spacing. In my mapping, alt.atheism is 0 and soc.religion.christian is 3, so their distance is 3, while alt.atheism and comp.graphics are only 1 apart. Ironically, the two religion-related categories would be treated as the least similar pair, even though their content is the most alike. KNN would use these false distances when choosing neighbors, and linear regression would assume each one-hot step in the code has the same effect. It would even imply that the "average" of alt.atheism (0) and sci.med (2) is com.graphics (1), which has no meaning.
3. I considered that one-hot binarization gives each category its own binary column, so no category is larger or smaller than another. Every pair of different categories is the same Euclidean distance (taught in the lecture) apart. For instance, the distance between [1, 0, 0, 0] (alt.atheism) and [0, 1, 0, 0] (comp.graphics) is $\sqrt{2}$, which is exactly the same as the distance to [0, 0, 0, 1] (soc.religion.christian). As a result, KNN no longer sees false similarities, and linear regression can learn a separate coefficient for each category instead of forcing a single linear trend.

AI used for this answer: Yes

If yes, how: Step-by-step guidance on running the code, extra code to show the category mapping and the shape of the binarized output, concept explanations of why integer codes create false order and distance.

External references (links):

- scikit-learn documentation, LabelBinarizer: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---